

ActInf ModelStream 011.1 ~ Hadi Vafaii: "Poisson Variational Autoencoder"

ModelStream | Hadi Vafaii | 1:10:03

all right hello this is active inference model stream 11.1 we're with hadhi vafai we'll be discussing the recent paper puts on variational autoencoder there'll be a presentation and a discussion so thank you to you all right thanks for having me um all right i'm excited to tell you about this new work that we did along with jake who's my postdoc mentor and decal who's a phd student in our lab so the big picture motivation behind this work is that we think we're going to understand the brain through this study of brain-like artificial neural networks but why do we think this way so all right think about a dream experiment where we recorded every single neuron in the brain of many animals doing complicated tasks in their natural environments we also have electron microscopy connectome of every single neuron and then you know learning dynamics is available to us as well we know everything about that those brains um this experiment is very difficult it's probably going to be impossible for the next i don't know 50 million years but if we had that data the challenge is we don't even know what to do with that kind of data set so the idea is that we're going to use anns as computational models of the brain to generate knowledge and theories and computational models to be able to analyze that kind of data whenever they're available so that's that's the motivation

the motivation but there's a challenge here so if you want to use anns to study the brain you better make them brain-like because you know as the cliché saying goes um all models are wrong but some are useful and the degree of usefulness of these anns is um is it corresponds to uh how brain-like they are in this particular work i'm going to show you uh what i mean by brain like it's it this word could mean many different things to different people but i have a very specific meaning in mind which i will tell you about and in this work our work focuses on uh models of visual perception so we're going to build an ann model that perceives visual stimuli and we're going to make it brain-like that's the whole idea so if you want to build um brain-like models you better draw inspiration from neuroscience and the idea is that we want to narrow down our search space because the space of all anns is huge and these are the specific there are three inspirations that we're going to rely on perception is inference rate coding and predictive coding i'm going to describe each of these separately first let's start by this all right so there's this idea that perception um involves two components there's this external component that is provided to us by the sensory data for example the photons that land on your retina and there's this internal component you have some subjective experience by living this world that gives rise to prior expectations that is combined with the sensory data to give rise to perceptions and this idea is not really new um you can trace it back to over a thousand years ago uh by alhausen he mentioned um vision occurs in the brain rather than the eyes and he was the first to discuss the subjective uh elements of perception and also more famously helmholtz said perceptions are best guess as to what is in the world given our current sensory evidence and our prior experience and the current sensory evidence is just

this external component and prior experience is the internal component let me give you an example so this is known as the Munsie room illusion we see this image and we immediately think that oh this must be a giant man and this must be a really short man but in reality um there's a bit you know different explanation so uh this room is designed in a very specific way to give rise to this illusion so we usually think rooms are rectangular like this dashed line over here but this room is not and the sensory data that is coming from person A on this corner is consistent with two different uh possible explanations one explanation that the room is rectangular and the person is short and the other explanation that the actual explanation is that the room is not rectangular the person is standing far behind person A and that's why we see it as short but you know because we have this prior expectation built in in our brain by you know living in this world where uh every room or most rooms we encounter are rectangular we immediately

just default to this explanation that oh yeah this person must be short so that's the idea we are prior expectations can actually trick us into perceiving things that are not true true and you can actually formalize this using mathematics of bayesian probability so here i'm going to introduce you very briefly to the idea of a generative model imagine you have some latent variable z that you can sample from the prior p of z and then condition some likelihood function based on whatever you sampled and then sample some observation x so x could be an image z could be some

abstract latent variable now within this context it's interesting to ask if i saw some observation some image x what are the likely causes underlying generation of x and that's the reverse direction so we have inference where we want to compute this bayesian posterior p of z given x which according to bay's rule is

proportional to these terms and i'll explain what they are so inference goes in the reverse direction from the image to the underlying cause of that image and now let's actually relate this back to that quote earlier from

helmholtz so he says perception is our best guess that's the posterior over here as to what is in the world given our current sensory evidence

that's the likelihood function and our prior experience and that's the blue over here so

um we think um or the theory of perception as inference states that that's really what's going on in the brain when

you show an image to the brain the brain thinks what latent configuration likely caused my current observation and in order to answer that we need to compute a posterior distribution over some latent causes given the observation this is the essence of perception as inference

next thing is rate coding and this idea is okay how do biological neurons communicate and store information the frequency we know that neurons produce all or non-action potentials or we call them spike and um the first evidence that neurons use uh so so rate coding is the idea that uh neurons encode information

in the rate of these spikes and this comes this goes back to all the way to 1920s where adrian and zoterman build amplifiers to be able to measure spikes and they show that um if you apply some stimulus to some

neurons

their firing rate increases and um in a in a proportional way to the magnitude of that stimulus and and this type of figure you can see it in any uh in many other different neuroscience papers um you know here they have some weight that is hanging from the muscle but you can actually you know um this this situation happens when you show a stimulus

and record from visual neurons or play a sound and record from auditory neurons and so on so the idea is that there's a baseline firing rate or number of spikes per second and then whenever some stimulus is shown to the brain the firing rate increases uh which encodes or represents the presence of that stimulus this is the idea of rate coding we're going to put this in our model

last uh the idea of predictive coding um the summary is that brains can be thought of as predicting predictive uh prediction machines this is the first sentence that this uh beautiful review by andy clark starts the the abstract starts by saying uh this basically and you know the summary of the idea is that the brain contains an internal model of the environment the brain uses this internal model to predict or anticipate sensory inputs and then when the sensory inputs actually um you know are received by the brain uh they're compared to the predictions and then there are some errors of course because even though we live in a predictable world um it's not fully predictable there's always some things that the brain is going to miss and those are the errors and these errors are propagated up the cortical hierarchy to update and maintain this internal model and you know this predictive coding idea uh in and perception as inference they have some um they're overlapping concepts but um but ultimately they're distinct and we're going to put all of these in our models here are some references if you are interested

in the point of view that if you combine these three ideas it narrows down your search space to this model which we call poisson variational autoencoder the poisson part comes from rate coding because in computational neuroscience there's a long history of modeling uh the observed spike counts from real biological neurons using the poisson distribution conditioned on some rate variable the variational autoencoder part

comes from perception is inference because if you actually formulate um you know what helmholtz said in in modern bayesian terms it turns out that these these architectures variational autoencoders exactly optimized for that uh loss so um that's why they're they're very very similar and then along the line down the line i'll show you how predictive coding is you know incorporated in this model

so okay but before um telling about poisson variational autoencoder i should tell you what is a dae or version autoencoder so the architecture um has two components in the simplest daes there's some input it could be an image and it gets encoded by this encoder network or you can refer to it as inference recognition these are you can use these interchangeably to you know infer some latents or infer some posterior distribution conditioned on x and then you can sample from the posterior and decode map this latent back to the observation and the idea is that when you are encoding this image into some set of latency um this encoding process should be good enough the the latents should contain enough information about the input image such that you can actually reconstruct it back that's the idea and that's the um what word autoencoder means because you're mapping some inputs to itself

and you know these can have different forms u_m but the loss function is u_m you want to you want to actually just uh do posterior inference and this is u_m what the posterior is p of z given x proportional to this and what we want to do is approximate this true but intractable posterior with some q which we are going to refer to this as approximate posterior and we're going to parametrize it with some θ and learn those parameters such that this term here is minimized this is the kl divergence of q and p u_m you can think of it if q is identical to p this term is going to be zero but if q is different from p it's going to be non-zero a positive value and actually if you just do some math and arrange some terms you end up with this loss i'm not going to derive this because this is pretty standard you can look up u_m you know any tutorial or blog post on vaes and they're going to explain this but what we end up with is this term called evidence lower bound or elbow which has two components so the first component is this expectation over q of $q \log p$ of x given z and if you think about this is like it maps some latence to observations and we want to maximize this term so we're going to refer to this first term as the reconstruction term because u_m it will have a high value if your reconstruction is is uh is good and there's this other term now another kl term emerges but this time between the approximate posterior q of z given x and the prior which is different than this and this we're going to just refer to this as kl term and it's called evidence lower bound because this relationship holds elbow is always less than or equal log probability this is called model evidence that's why it's evidence lower bound it's a lower bound and model evidence and if you actually multiply elbow by minus one it turns out that this is exactly the variation of free energy which is larger than surprise which is negative log p and this variation of free energy is exactly the same thing as as it appears in fristonian free energy principle so okay the e loss we're going to maximize elbow or equivalently minimize variational free energy so now let's talk about u_m how do you actually let's get a little more specific so we're going to build a gaussian vae u_m i should have mentioned that you know that the first of probability distributions all the prior posterior likelihood that's up to the practitioner you have you know u_m you have freedom to choose any distribution you want but you know most of the literature in vaes is actually just using gaussians for those distributions our contribution is we're going to replace gaussians with poson but let's actually just understand what gaussian vaes are about okay so both uh prior and posteriors are gaussians the prior is chosen to be a fixed standard gaussian with zero mean and unit variance and this is again another choice you can actually learn the prior but in standard vaes there's no learnable learnable parameter it's just a fixed distribution and the posterior is parameterized with some mean and variance that is produced by the encoder network and if you just compute the kl term this is what you get again i'm not going to derive this because pretty standard at this point and and from this closed form solution to the kl term you can see that it will be zero then μ is zero and σ is one that means the posterior collapses the prior this is known as posterior collapse so if they're identical of course their divergence is going to be minimized which is equal to zero and u_m operationally this is what happens when you have a gaussian vae so you give it an present an image to the encoder and it spits out some mean and variance vectors and u_m it's red that means it's parameterized

by the encoder network and so on and then you construct this posterior distribution this is the same as q of z given x and then you sample some latent from it this distribution and then you map this sampled latents back to its reconstruction and this is what happens during the forward pass but you want to be able to compute the loss you know you want to take this $\hat{x}$ and compare it to x and you know compute gradients update your parameters but it's impossible to do it if you actually just sample from this posterior and to circumvent this problem the original vae paper introduced the reparameterization trick so instead of sampling from this distribution which is your posterior you can sample noise from some unit gaussian and then compute z as $\mu + \epsilon \times \sigma$ it turns out that sampling from this distribution and this operation they're exactly identical but this is better because now you can actually compute gradients and use back propagation to learn your network's parameters so what we're going to do is we're going to replace these gaussians with poisson because of the rate coding idea that's the inspiration and in order to make this vae work we need to do two things first of all we need to introduce an equivalent reparameterization trick for the poisson case we also need to compute the kl term we're going to do both of those in the next few slides all right so as i mentioned now we just take the prior and posterior we're going to replace gaussians with poisson and so gaussians need both mean and variance you have to define both a mean and a variance to have a gaussian distribution but for poisson you only need one number that's just the rate parameter and this is the pdf of a poisson distribution give me a rate and i have this distribution so we have this prior rate and the posterior rate which depends on x and this could be you know any general you can learn the prior rates you can know output any some generic posterior rate but here is where we add the predictive coding assumption we say that okay look let's call the prior rates just r and interpret those as some representation units that are just maintaining a prediction or anticipation of what is coming what kind of sensory information is coming their way okay and then let's assume that the posterior rates are actually just the modulation of those existing rates so we just say take r and the encoder only computes some quantity we call it Δr that is a function of the specific stimulus shown to the encoder and then you obtain the posterior rates by multiplying these two together and this is just an illustration of the idea so you have this input it is processed by some encoder to produce Δr and it is understood that this Δr is going to interact multiplicatively with existing prior rates and then once you have this multiplication you're gonna now you have your posterior distribution this is your approximate posterior and then once you sample from this you're gonna get discrete spike counts because now you have Poisson in the gaussian case you would get continuous values but now we're gonna get discrete integers

which is

illustrated over here like for example let's say we have five neurons in the latent space we sample from this posterior

so here we have a vector of one three zero zero one so we take that vector we pass it through the encoder and the encoder learns to map it back to whatever image that was shown and these are all learnable parameters

so the decoder network the encoder network and the prior rates they're all learnable parameters

so i want to emphasize there's like a bunch of difference between Poisson and gaussian VEs the first difference is okay it's Poisson it's not gaussian the other difference is prior is learned and and then the last difference is that the these prior rates are parameters that are shared between both the prior and the approximate posterior

Now let's talk about the Poisson reprimatization trick we want to be able to when we sample event counts or spike counts from this Poisson distribution we want to be able to um differentiate through this operation so how can we do that in order to show you the um the intuition behind our reprimatization trick let's actually focus on the process that generates event counts so if z is some event count that is distributed according to Poisson we know that um there are some inter event intervals that are distributed through according to this exponential distribution what Δt is is that suppose one event has happened um how long do you have to wait until you observe another event those wait times inter event intervals they're distributed according to this exponential distribution with parameter λ and and this is just a well-known property of Poisson distribution we're going to use this we're going to exploit this fact to you know introduce our our trick all right so now suppose we want to actually just run this process um sample many times from this exponential distribution

and then um in a finite window of observation time of one second we want to see how many events would we get

so when we sample from this exponential maybe we get first uh time wait time is Δt_1 so we wait this much until we observe the first event we sample again it takes this much time to observe the second event again third event but the fourth event doesn't really occur between zero and one which is our previously defined observation time therefore this doesn't count so we have a total of three events because the sum of these uh three Δt 's is less than one so um sampling these from exponential and then counting them like that like i like we did is equivalent to just sampling from

Poisson and getting like number three as our spike count so we can actually just sample these numbers in parallel from the exponential distribution

we compute their cumulative sum uh which is visualized over here and we can just apply the thresholds and say

all right whatever number of cumulative sums that is less than one let's take that as our um spike count and here which means we're going to have three events and and this is not going to count and any other Δt 's after this is not going to count because it's just going to add over here it's it's it's uh extends beyond our wait time so again we have total event counts of three it turns out that you can just take this and turn it into an algorithm um so the input to the algorithm is just some rates that are produced by the

encoder network then we have uh we have to specify how many exponential samples that we want to generate

there's a temperature that that controls the sharpness of the thresholding so if you want to threshold put a hard threshold of like anything less than one uh that's not going to work you have to um approximate that with some sigmoid uh that's what we have over here okay so that's our reprimatization trick and um and we hope that this this trick you know it can you know be used beyond the realm of VAEs for example in spiking neural networks uh people deal with this issue all the time you know it's really difficult to work with discrete stochastic variables so we're hoping that this algorithm will find applications elsewhere as well

but now let's talk about um the loss function derivation as a reminder we have our prior given like this and the posterior given like this and our goal is to compute the KL term

so let's first start by just the definition of the KL term for any distribution q and p the KL between q and p is this expectation over q of log of q divided by p so so far this is just general definition but now we're going to plug in the actual poisson distributions that we have

so just put that over here we define λ to be r times δ r it's just our posterior rate and this is uh what Poisson distribution looks like so i just replace q and p and immediately we see that z factorials cancel and we can actually just write this term as follows it's just log of λ over r to the power z plus uh log of e to the minus λ plus r and then z comes down λ over r is δr so you're going to just end up with $z \log \delta r$ and log of e to the something is very simple it's just you know you bring the exponent down and in this entire expression the only thing that that has an expectation is z everything else is a constant so it comes out and the expectation of z uh over q is just it's the mean and the mean of Poisson distribution is just the rate so z is going to be replaced by λ which is the rate of q and that's what we get here so r minus λ plus $\lambda \log \delta r$ r and again we just take r take it out and it's like one minus δr plus $\delta r \log \delta r$ so we see that nicely um by having you know by virtue of choosing this kind of predictive coding like assumptions uh our KL term nicely factorizes to a term that depends on prior and another term that depends on the modulation that comes from the encoder and then we define this whole thing in parentheses to be f that's f is one minus this plus whatever and if you plot it it looks like this so δr is by definition always positive it's greater than zero and um if δr is one it says like okay do not modulate anything let's just keep the prior rate then this f becomes zero and if δr is really large you pay the cost of you know it really increases the magnitude of f of y

so okay so this KL term was just for one neuron um if you have k statistically independent neurons k being the dimensionality over our latent space you can just easily extend this derivation to see that the KL term is just the sum of of these terms and now this is the interesting part we're going to interpret this term as a metabolic cost term the reason is r you know the prior firing rates they are positive values by definition and we ended up with some term f that also is positive by definition so you have a positive term multiplied by another positive term so this thing whole thing is is just a positive value and we want to minimize it and we can minimize this for example by just putting zeros in

all of our prior rates like all of these neurons are silent they're not doing anything and and that's why we are interpreting this as a metabolic cost because you know producing action potentials um is is a very costly metabolically costly operation so the neurons in the brain they don't want to fire action potentials unless it's absolutely necessary to encode some information or communicate something that's why this is very interesting that it just comes out of the math and if you just put everything together this is just the the loss of boss on dae the first term is the reconstruction term um it's just the L_2 norm of x input minus reconstruction this is like decoder network applied to one sample which is drawn from the posterior uh so this is the reconstruction plus this term which is comes from the kl term and this is very curious because now we have some some term that says like reconstruct the image and another term that says do it while uh optimizing or minimizing neural resource use and this is reminiscent of sparse coding which i will introduce here so in 96 old thousand and field introduced sparse coding they said okay let's imagine there are some images x and some latent variable z um or like let's say neural activation z in their case um and let's assume that we represent these images as a linear sum of some basis elements from a dictionary ϕ while we're representing these images let's also try to minimize uh the magnitude of neural activity which is this L_1 term over here here and just um you know minimize this whole loss that'll give you sparse coding of course because you know if you if you apply this term it's going to push a lot of these uh neural activities to zero it's going to give you a sparse activity vector and and this really resembles what we get from pvae and in order to actually make this connection more clear we just take this general decoder term that appears in the loss we also assume we have a linear decoder just like sparse coding and it turns out in that case you have a closed form solution for the loss um i'm not going to derive this it's in the paper if you're curious but you know there is a closed form solution um in in the actual you know full loss there's this expectation value which you can compute it gives you this final loss and again we see this this uh correspondence nicely so there is one term that says um faithfully represent images with some linear decoder and then there are two terms here actually it's as opposed to just one there are two terms that push your firing rates to low values so this diagonal of $\phi^T \phi$ is positive by definition and λ is positive so this thing needs to minimize therefore it forces λ to minimize and again just a reminder that this is coming from the kl term and we interpret it as a metabolic cost term okay so let's just pause a little bit here we started from all those um three neuroscience inspirations we said we want to build a model that does um you know perceives visual inputs we want to build in um perception as inference rate coding and predictive coding all of those in the model it gives us it um it gave us the the boss on vae idea we derived the loss and we saw that uh it resembles sparse coding so it's just uh gifted to us by the theory now we want to test this we're going to say okay the loss resembles sparse coding but let's actually just train some models and see if we get sparse coding like results that's what we're going to do

next the experiments oh yeah this is okay so this is the architecture of um linear pvae so you have again some image is encoded into Δr multiplied by r to give you this posterior and importantly now the decoder is just a linear layer that's what this is about and another uh important assumption that is central to sparse coding is that you have to have an over complete latent space so you have many more latent dimensions or many more neurons than you have pixels so m is the number of pixels of the input image k is your latent dimensionality now we're going to train this model and compare it to sparse coding

but also um in order to understand what uh components of the pvae might you know um you know reproduce sparse coding we're going to compare it to alternative vae models so we have discrete vaes we have pvae over

here which is what we introduced and we're going to use um categorical vae uh and there are continuous vaes gaussian vae and laplace vae

and the reason we chose laplace to to include in in this uh in this table is that people have shown before us that if you have a laplace distributed prior you can also get sparse coding like results so we want to test which one of these models is actually uh more sparse coding like if you will and then we're going to compare to actual you know um very well established standard sparse coding algorithms one is the locally uh competitive algorithm from 2008 it's a biologically inspired algorithm that uh uh optimizes the sparse coding loss and another algorithm is uh ista these are both very standard and uh well established uh as well as the gaussian so you're going to compare to a bunch of models

a bunch of models all right so if you train this pvae on linear pvae on natural image patches

this is what we get these are the decoder weights and um if you're not if you're in neuroscience in in visual neuroscience you already know that these are you refer to these are like the borer patches and and and uh these resemble the selectivity in the primary visual cortex of monkeys and and mice and and humans so so this is very biology like and for the first time these types of gabor patches they they emerge from the sparse coding models which we are we were able to reproduce with pvae and if i show you the other models it's not actually trivial but let's go from left so we have gaussian vae over here i'm not sure if you can see but um there's a lot of noise and there's only like last three rows uh look you know they they look like pca and that's expected because a linear gaussian vae is theoretically equivalent to probabilistic pca now we have also laplace vae down here um there are some gabor's that emerge from the laplace vae but again there's a lot of dead neurons which i will define what dead neuron means and i'm going to show how we quantify them but you know for now let's just visually inspect these learned decoder rates this is the pausanne vae and this is the categorical vae categorical vae is discrete so it also learns some gabor's but it's very limited so um that there's a lot of noisy elements and certain spatial frequencies are missing and certain orientations are missing as well from the categorical vae and finally in the last column we have these results from the standard sparse coding algorithms these are the best you know gain in town if you want to learn sparse coding like uh dictionary elements and we see that among the vae models uh poisson vae actually is the is the like most similar one to these standard sparse coding algorithms so in conclusion because both uh the loss

function of a pvae just emerged to resemble sparse coding and also empirically we you know tested it and also learned sparse coding like results we conclude that it actually contains sparse coding as a special case why because you know we we made a linear decoder assumption so it's like pvae can be um general like it could be anything could be non-linear encoder non-linear decoder but if you make the decoder linear and make the latent space over complete you're gonna recover sparse coding like results okay so here's um how i quantify number of dead neurons this is just uh let me just first define what the posterior collapse is this is a big kind of prevalent issue in daes when a latent dimension stops encoding information it's going to be always the posterior in that latent dimension is going to be identical to the prior so there's no deviation the kl term is very small or zero and that's how we measure dead neurons and here this dashed line shows the gap between the post the the kale of dead neurons

and active neurons that's how we quantified and we found that for the pvae uh across three different data sets it consistently had much larger number of active neurons compared to laphas and gaussian baes so it kind of automatically avoids this posterior collapse issue because of discreteness that's the the main cause of this observation okay so we want to actually do some other experiments um so far we just saw that if you start from those neuroscience inspirations you get a model that also uh you know recovers or reproduces some other neuroscience like uh models result like sparse coding but now let's just be a little more general we want to evaluate pvae in a general representation learning setting that's what this is about all right so we want to test these unsupervised representations on a downstream classification task and we use mnist for that purpose we're going to compare all those four vae models for this task

the approach is that after you know the training is done we extract encoder representations this means we push mnist digits from the validation set through the encoder we extract the representations and we're going to

do things with those all right and then once you have these representations sample a limited number of supervised labels so in this you know they have labels we know each image of a digits has a label so it's a supervised data set um but we're going to look at sub samples like very small samples um of of supervised subsets and and the reason is that you know um having labels is expensive so if a model can achieve good performance with a small subset of supervised labels is is a good model that's the idea then we're going to apply k nearest neighbor classifier um to to classify these these uh digits and the idea of using knn is that it's a non-parametric model so you don't need to learn anything on top of these representations and the output of a knn classifier directly depends on the geometry of representation so it's going to tell us something about if there's something fundamentally different about the geometry of representations in one of these vaes like pdae versus the others and then as you know as measure of success we're going to report classification accuracy this is the results um you know we have different latent dimensionalities we have different models there's a lot of details that i'm going to skip but let me highlight just one thing so let's look at this number so for 10 latent dimensions with only 200 samples the pvae achieves accuracy of around 82 percent but gaussian vae

requires

five times that number a thousand label samples to achieve a similar performance in this sense we claim that pvae is five times more sample efficient than the standard gaussian vae and you know across the board it just outperforms all these other models which shows that it learns useful representations that can be applied in downstream tasks finally let's quantify sparsity uh we're gonna use this measure from viny and gallant this is called lifetime sparsity um z_i is the response to i -th stimulus number of spikes in the poisson vae and n is the total number of stimuli um just the intuition about this formula is that um if if if a neuron only responds to a single stimulus and doesn't respond to any other stimuli the sparsity score is going to be one and if the neuron or the latent dimension responds to all stimuli uh equally then the sparsity score is going to be zero so zero means not sparse at all one means uh very sparse and use that this is the the y-axis this the the lifetime sparsity of these different um vae models and on the x-axis we have the reconstruction error in mean squared error um we see that poisson gae is very sparse but because of that sparsity it's uh has larger reconstruction error compared to let's say gaussian which is down here and it turns out if you apply some some relu um to the gaussian it's going to increase the sparsity but uh not as much as poisson a sparsity and then this curve over here came from fitting a sigmoid to a bunch of poisson gae's that were trained with different beta so if you remember beta is a parameter that um uh kind of changes the trade-off between the reconstruction term and the kl term so if you increase beta you're going to get much sparser results but uh at a cost of larger reconstruction errors and if you if you decrease the beta you're going to get less sparse results but but better reconstruction so that kind of traces uh rate distortion trade-off finally um one thing that we notice is that there is still a large amortization gap for the poisson gae and i'll explain what that means so if you compare poisson gae with those standard uh sparse coding algorithms including lca which is this dark gray and ista which is this light gray we see that they achieve the same level of sparsity roughly uh but poisson gae still has um a larger reconstruction error but you know what what happened here like what is why is um lca much better than poisson gae so to test this we thought that you know maybe uh the dictionary elements learned by poisson gae are are pretty good but the actual inference is is not very good the encoder architecture is not doing a good job in inferring the activations so what we did is we took the plus on gae and we took um sorry the the dictionary elements learned by the posan gae even applied lca inference on those and that gave us this these uh purple dots which we also fit a sigmoid to and it turns out that you can actually just close the amortization gap by applying a better inference which in this case is the lca inference to just uh bring this over here so the conclusion from this slide is that the posan bae learns reasonable dictionary elements but its encoder or the encoder architectures we tried in this paper they weren't adequate and there was this large amortization gap that in the future work we hope to to kind of close this gap but you know building better encoders maybe iterative encoders and so on okay let's just conclude um so we started from these three uh inspirations perception is inference rate coding predictive coding it gave us this architecture and um it's interesting for many reasons because you know posan dae encodes its inputs in discrete spike counts therefore it's

more biologically realistic and this metabolic cost term emerged in the model objective for free we didn't have to put it by hand or anything and it actually revealed this connection to sparse coding which we verified empirically and then from a theoretical perspective uh it kind of unites all these different ideas like predictive coding rate coding sparse coding under the umbrella of bayesian inference which is nice and then the takeaway message is that you know if you draw inspiration from centuries of neuroscience research combine it with modern machine learning you might get a model that's more brain-like than alternatives so

and it outperforms alternatives in key aspects like sample efficiency

you know all right this is uh all i had for the slides um now if there are questions

to go over there or maybe i can show the code thank you that was awesome yeah how about the code and meanwhile for people who are watching write any questions and then i'll ask them thank you

all right um yeah so i actually haven't prepared anything specific to say about the code but i'm just gonna riff through you know what what exists um hopefully some some people might find this inspiring okay so

this code is actually not online right now it's not available but because the paper is under review but

once the paper gets out um i'm gonna make this code publicly available so that maybe the more more important

uh part of the code is that you go in the code in the vae you have vae.py which has all the implementations of different vaes that we discussed in in this talk so there's a base vae that um has all these different

functions it encodes um inputs you know you can have a convolutional encoder you can have a linear encoder

mlp encoder it's all implemented you know same with decoder you have convolutional linear mlp

it computes the loss which is just the nsc and you can also compute the exact loss when you have a linear decoder um so the kl loss is not implemented because we're going to have specific instances of vaes like

you know poisson vae gaussian vae etc and they each have their own kl loss so that's that's not implemented over here in the base vae and so on there's there's a bunch of you know different

functions so let me just show you the actual implementations

so poisson vae is over here um

in the forward direction you have this this infer function that accepts x which is in the image

temperature and it computes so the self dot log rate this is the prior rates these are parameters

that are learned along with every other parameter and you have this log r um then you have log dr dr

is just a delta r that's the output of the encoder where you encode the image as an output you get log dr

and then um skip skip this part then you compute you you make this distribution and and just to remind yourself that dist is just poisson which i will show you later it's defined somewhere else and then you

you make this distribution you provide the log rates this is the nx if you remember from the

the reprimatization um slide you have to provide how many exponential samples you're gonna draw which

you know it's it's adaptively computed somewhere else in the code i'll tell you where later you provide

the temperature and then you have a distribution which you know this is going to be the infra function

provides you with the distribution that's your approximate posterior and then you sample from it and get

spikes you decode those spikes that's your reconstruction that's it there's other functions that i'm not going to go but maybe like let's have a look at the kl term so um you can easily you know understand this you know if you understood the the slide deriving the kl term in theory then you would understand this term maybe i'll i'll show you quickly the distributions so we have also this

base distributions which contains our implementation of the poisson distribution and the reparameterization trick for it

um there we go it's here yeah so you provide it with log rates um in the initialization you provide the temperature which is the thresholding temperature number of experiment number of uh exponential samples

and the clamp is just basically says if the rate is larger than e to the this that you know it applies a soft clamp uh which is e to the this value so it applies an upper bound on the rates this is for the you know stability of training it's like uh not very important but the important part is here this function that's our reparameterization algorithm okay so first of all you

you initialize this exponential distribution in this init function so you get log rates apply the soft clamp and then you define rates as this just uh exponentiate log rates plus this really tiny value so that's not zero um because if it's zero then this exponential is going to raise an error it has to be slightly above zero so you've defined this exponential distribution which takes the rate as input and here i'm just borrowing from pytorch so this uh if i go up i'm importing uh it's not over here i imported somewhere else but basically

um where is it

this is the torch distributions i'm just importing it as disks so this exponential is exactly pytorch's implementation all right once we initialize this self.exp object then we can actually just perform pre-parameterized sampling so x is just uh the first you know those all those delta t 's that i showed in the slide these are your x 's and you have to sample n times and then you perform this cumulative sum to get the times and then if temperature is greater than zero you have this soft indicator which is just sigmoid you know if it was temperature was zero then you would have this hard indicator which is just uh times less than one if you take this operation if you make it continuous you will get this basically that's the idea once you have this indicator function then you can sum them in the first dimension to get your spike counts all right so that's that's for the poisson but i want to also mention this um continuous vae because this is a more general implementation that you can use for any you know any vae that uses a continuous distribution like in our case we used gaussian and laplace but you can actually take this implementation and implement you easily build your own continuous vae like kaoshi or any other distribution you like as long as you can you can reparameterize it which a lot of pytorch you know implementations they have this function called `r_sample` which is requirements a sample but again the you know this is a very similar idea you encode some features take some x encoded into this feature h

h and then because for these continuous vaes you need both the mean which is location and the variance which is log scale so you just divide h into two parts you get these parameters and then you you know construct this posterior distribution and then return it that's what infer does it always gives

gives you the posterior distribution once you have the posterior distribution you can sample or reparameterized sample to get your latent ignore this part you know if you can actually just apply an activation function this is the gaussian cross layer that I kind of skipped that didn't really talk about it but that's that's there and then you can decode and y as your reconstruction so so this this continuous vae implementation is fairly complete you can in order to build your own that's all you have to do so you just say okay I want to make a gaussian vae I'm gonna inherit continuous vae object I'm gonna say okay so that this should be normal that's it and for laplace again you inherit the continuous vae and just say the distribution should be laplace and you can do this for any other continuous uh distribution maybe um that you know these are these are pytorch's implementation so any continuous distribution uh implemented by pytorch that has reparameterized trace sampling uh you can use this to build your own vae and then we have this categorical uh which I'm gonna skip but you know we use the again pytorch's implementation and okay so so this this um this module defines vases and um there's another module that trains them so let me just briefly go over that again there's a bunch of code that just defines base trainers there's a lot of details here that maybe not very interesting but you have this trainer that accepts all these different vae types gaussian categorical laplace and it goes through this is just one iteration um there's a validation function all of that and it's fairly complete so there's a bunch of um arguments that you have to provide to be able to run this code but because I spend a lot of time to make it intuitive it's going to be just a single line if you want to train your own pvae um you can just run uh that with a single line like you know train vae you specify uh the architecture in one string you specify the data set and um and the gpu that you want to train on and then then you can actually do that if you want but yeah this this code is not available right now when I put it online I'm gonna make some documentation and you know put some scripts that make it easy and intuitive to just train your own model if you're interested there's a lot more but I think at this point I'm gonna stop here there's there's a bunch of other stuff uh that is secondary yeah awesome okay while I um crop back the screen maybe just let's start with how did you get to this problem were you pursuing poisson distributions and this came up or how did you kind of um in your research path come to this approach oh so the the real initial motivation is just you know um we did another paper um last year on a gaussian vae that was hierarchical and we found that that was um we trained that gaussian vae that gaussian hierarchical vae on a motion data set and we found that um it learned latent representations that were very aligned to real biological neurons from the monkey brain so we have another data set uh from um the data set was they showed some moving dots to monkeys and they recorded from their MT which is a brain region that's you know that is responsible for motion perception anyway we found that these vases actually have a really high chance of being brain-like brain-like and that gave us motivation to think about okay how we can make them even more brain-like so gaussian vases just encode inputs in continuous values that are not you know constrained by any in any means but we know that that's not how brain works we know that neurons produce spikes that's how they communicate so we said like okay vases have potential but how how can we make them better that was really the

initial motivation awesome well just to kind of recap a little bit of how i saw this often parameters in biological or cognitive modeling are modeled as continuously dynamically varying because there are certain simplifying assumptions however the empirical data be it neural spiking or events or like ants foraging how do you bridge the discrete events to the continuous and the um mathematical equivalence which is well known between the event arrival distribution with the person and the waiting time i saw that as a way that you implemented with the statistical methods to and also in doing so generalized like what was relevant for the base model for continuous or continuous or discrete and then how can that be articulated with the statistical packages so that there's good course handles to explore these learning settings with the i think that's actually very interesting and and deep points you know um even though neurons communicate with sprites or discrete action potentials when we're modeling them you can actually think as um you know the actual underlying dynamics could happen in a continuous space like for example we have these log rates you know those log rates are continuous values between minus infinity and plus infinity really i mean even though in reality it doesn't go that far but they're defined in if you have n neurons log rates are defined in $\mathbb{R}^n$ right and then uh you can assume that the model is just doing some trajectory like the brain is doing going through some trajectory in that log rate space and in each given time you can actually take this that log rate and then map it to some posterior by computing the exponential of that to get the rates and then draw samples and in that sense you can actually see that a continuous underlying dynamical system can be mapped into a discrete observation through through pausanne and also that um every time point in the dynamics of the brain also corresponds to some posterior inference so these concepts i think can be nicely you know reconciled with this view yeah that was great and to kind of connect it to active inference and a little bit of the precursors there with spm statistical parametric mapping the whole setting is there's underlying neural activity hidden state external state and then there's noise plus signal representing a static mapping between the hidden and the observable what you had is x and z so that's like the partial observability situation and then that's the experimental situation but you don't get to observe neural rates so like in the spm textbook and the kind of modeling you model it with the predator prey type models or with dynamical attractors but you stay in the purely dynamical and then that doesn't let you get to the rate coding but also people have integrated those models however in more of a um two-step process rather than the one-step unified objective of the autoencoder yeah so that's pretty interesting so because rate is in a sense the rate itself is a latent variable right we never observe it we it's a it's a made-up concept really by the

theorists we say okay there if there is an underlying rate and that is producing these observed spikes you know in the brain then then you can actually try to infer the rate by observing many spikes over many trials

and so on so in that sense here rate itself is a latent variable um but that that maybe you shouldn't go there because that like adds another layer of complexity well one question i had about rate was if you're going to model it you kind of can't get away from it as a underlying dynamical variable because there's going to be variability in Δt

variance amongst arrival time so maybe it's a metronome but in the non-metronome case there's going to be kind of like

faster and slower intervals so that's kind of the common filter question which is or or a rima or any time series ultimately how smoothly should you interpolate that like with the splines with what method will the data kind of have a relationship with a curve that has the smoother differential properties that ultimately allow the closed form solution and the simple simple algorithms

makes sense

um okay i'll see if anyone asks a question but um just one question now and then i'll ask maybe one or two more

um like where do you see this being useful slash applied now so we are actually currently building um based on this observation that there was a large amortization gap between pulsion vae and this kind of relatively old sparse coding algorithm from 2008 the one conclusion we drew from that is that you know we need iterative inference so right now the inference is just one pass you take one image you infer something to compute Δr and then you just get the posterior but now suppose you have a sequence you have a video rather than images and you want to just keep iteratively making your inference better and better such that your um let's say your posterior at time t informs your prior at time $t + 1$ that's like very essential to bayesian thinking so we're kind of right now uh developing that model and our hope you know very long-term hope is that you know when we add that component and maybe we add

hierarchical structure to the latent space then we get like something that is truly brain-like and it reproduces what we've observed what we've learned about visual processing in the last like you know 50 years

so if you get that model then you can perform in silica experiments in it learn something about brain like visual processing and so on so that's kind of roughly the the trajectory we imagine awesome is there anything else you want to add otherwise i think this was amazing on the theory and really seems useful with the sample efficiency all these other features

yeah um i guess that's it yeah thanks for having me it was really fun awesome okay i'll just read one comment from someone wrote great work thanks to active and agha hadhi for sharing looking forward to find out what a different inference scheme could do the results and the effect it could have on the metabolic cost okay yeah thanks for the comment yeah cool that in mind all right thank you see you all right thank you bye

you